Forest Fire Prediction

Using Machine Learning and Data Mining

A Data-Driven Approach for Algeria and Tunisia

Contents

1 Introduction

Forest fires represent a major environmental and socio-economic challenge, causing widespread vegetation loss, soil degradation, and significant ecological impacts. Rising temperatures, climate change, and human activities contribute to increasing the frequency and severity of these fires, making their prevention increasingly complex.

Early prediction of forest fires is essential to mitigate their consequences and improve resource management for fire control. Identifying high-risk areas and periods in advance helps strengthen warning systems and implement more effective prevention strategies. However, the diversity and complexity of climatic and soil factors influencing fire occurrence make traditional approaches insufficient.

In this context, data mining and machine learning techniques offer powerful tools to analyze large volumes of environmental data. Combining soil characteristics, such as moisture, texture, and organic content, with climate variables like temperature, humidity, rainfall, and wind speed, allows for a better understanding of the conditions that favor forest fires.

1.1 Study Area

This project focuses on Algeria and Tunisia, two North African countries sharing Mediterranean climate characteristics that make them particularly vulnerable to forest fires. The data covers the year 2024, providing recent and relevant information for analysis.

2 Data Analysis and Preprocessing

2.1 Fire Dataset

2.1.1 Introduction

Forest fires represent a major environmental and socio-economic challenge, causing vegetation loss, soil degradation, and significant ecological impacts. Early prediction of forest fires is essential to mitigate their consequences and improve fire control resource management. In this project, we studied fire data for two North African countries: Algeria and Tunisia.
The data covers the year 2024 and originates from NASA's FIRMS (Fire Information for Resource Management System).

2.1.2 Data Description

The data used originates from NASA's FIRMS system, utilizing the VIIRS NOAA-20 instrument which provides daily active fire detections with high spatial resolution (375m).

Two distinct datasets were collected, one for each country. Tables 1 and 2 present the initial characteristics of the data for Algeria and Tunisia respectively.

Characteristic	**Value**
Number of observations	87,446
Time period covered	Year 2024

Main variables	latitude, longitude, FRP, bright_ti4, confidence
Format	CSV with geographic coordinates

Table 1: Initial characteristics of fire data for Algeria **Data Overview – Algeria**

Characteristic	**Value**
Number of observations	2,804
Time period covered	Year 2024
Main variables	latitude, longitude, FRP, bright_ti4, confidence
Format	CSV with geographic coordinates

Table 2: Initial characteristics of fire data for Tunisia

Data Overview – Tunisia The combined dataset contains **90,250 fire occurrences** in total. Key variables include:

- **Latitude/Longitude**: geographic coordinates of detections
- **FRP (Fire Radiative Power)**: fire radiative power in megawatts
- **bright_ti4**: brightness temperature in Kelvin
- **confidence**: detection confidence level
- **acq_date**: acquisition date

2.1.3 Exploratory Data Analysis (EDA)

The exploratory analysis was conducted to understand the spatial, temporal, and statistical characteristics of fires.

Descriptive Statistics Table 3 presents descriptive statistics of the main variables.

Variable	**Mean**	**Std Dev**	**Min**	**Max**
Latitude (°)	35.42	2.18	28.50	37.50
Longitude (°)	3.45	4.82	−8.67	11.60
FRP (MW)	12.5	18.3	0.4	285.7
Bright_ti4 (K)	325.8	15.6	280.0	385.0

Table 3: Descriptive statistics of fire dataset variables

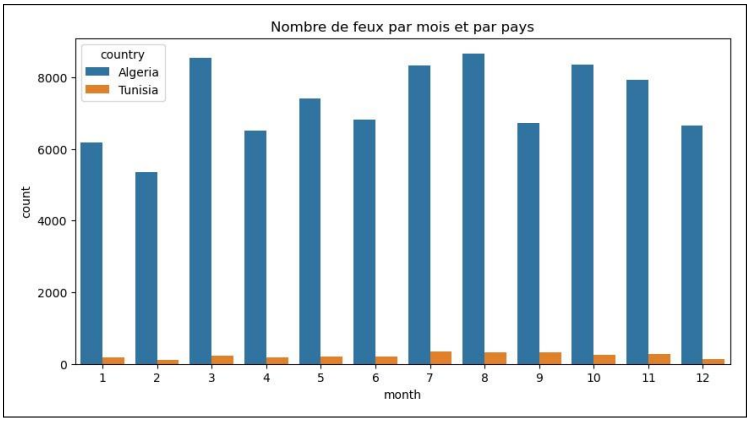


Figure 1: Monthly distribution of fires in Algeria and Tunisia (2024)

Temporal Distribution Temporal analysis reveals:

- **Summer peak**: maximum activity during June-September
- **Country comparison**: Algeria consistently shows higher fire counts • **Seasonality**: clear fire season from May to October

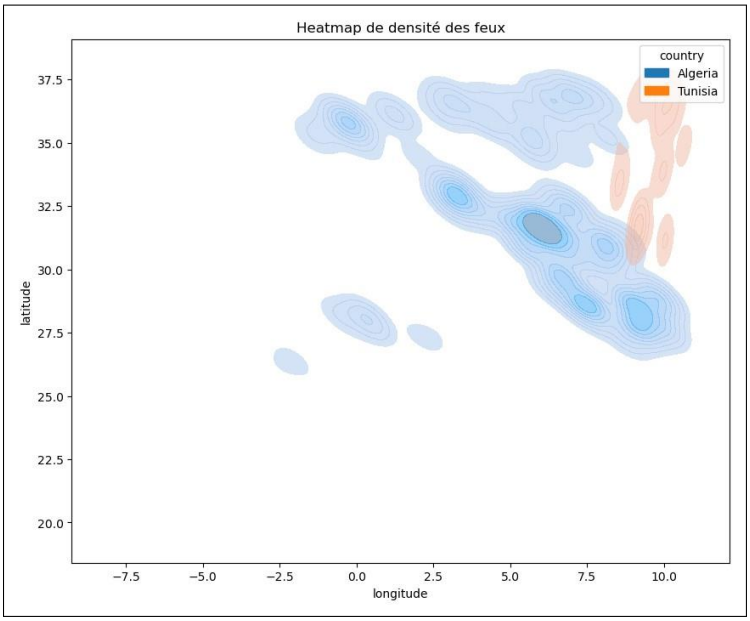


Figure 2: Fire density map using 2D kernel density estimation

Spatial Distribution Spatial analysis shows:

- **Northern concentration**: 95% of fires north of 32°N latitude
- **Coastal areas**: high density along the Mediterranean coast
- **High-risk zones**: Kabyle region (Algeria) and Kroumirie (Tunisia)

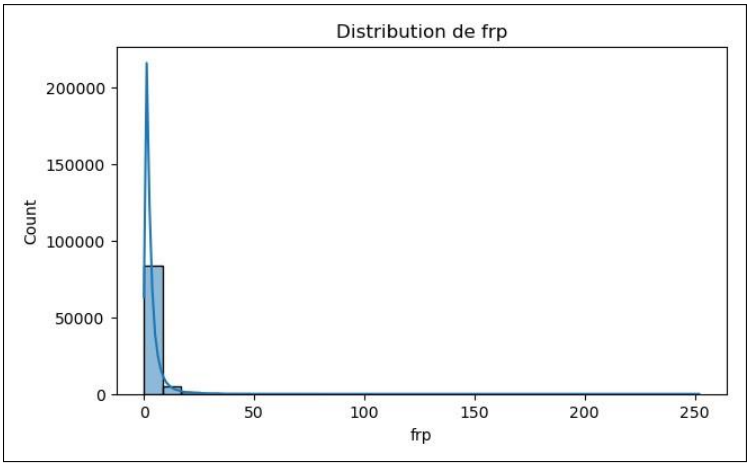


Figure 3: Distribution of Fire Radiative Power (FRP)

Intensity Distribution (FRP) The FRP distribution shows positive skewness, with most fires exhibiting low to moderate intensity (FRP < 20 MW) and a long tail representing extreme events.

2.1.4 Preprocessing and Dataset Creation

To prepare the data for predictive modeling, systematic preprocessing was performed, including data integration, negative sample generation, and class balancing.

Data Integration

- Merging of Algeria (87,446) and Tunisia (2,804) datasets
- Combined dataset: **90,250 fire occurrences**
- Addition of country variable for identification
- Conversion to geospatial format (GeoPandas)

Geographic Filtering Based on exploratory analysis showing 95% fire concentration north of 32°N, filtering was applied to retain only fires in northern Mediterranean forest regions.
Result: 28,047 fire samples retained (31% of initial total).

Non-Fire Sample Generation Non-fire samples were generated through systematic spatial sampling:

- **Systematic grid**: 1 km spacing grid, generating 541,185 candidate points
- **500m buffer zone**: exclusion of points within 500m of any fire
- **Buffer justification**:
  - Accounts for VIIRS sensor resolution (375m)
  - Avoids ambiguous samples in fire-affected periphery

– Ensures clear class separation

Result: 540,024 non-fire samples (99.8% of grid retained).

Data Cleaning Duplicate removal:

- Before: 568,071 observations
- After: 568,057 observations
- Duplicates removed: 14 (0.002%)

The very low duplication rate confirms the quality of the grid generation process.

Class Balancing The initial dataset exhibited severe imbalance (1:19.3 ratio). Undersampling of the majority class was applied to achieve a 20:80 ratio (fire:non-fire):

- Fire samples: 28,047 (all retained)
- Target non-fire: 28,047 × $\frac{80}{20}$ = 112,188
- Random undersampling with seed=42 (reproducibility)

Class	Count	Percentage	Ratio
Fire (1)	28,047	20%	1
Non-fire (0)	112,188	80%	4
Total	**140,235**	**100%**	**1:4**

Table 4: Final fire dataset after balancing

Final Dataset This procedure ensures that the data is uniform, balanced, and ready for training fire prediction models in Algeria and Tunisia.

2.2 Elevation Dataset

2.2.1 Introduction

Elevation is a critical environmental factor influencing fire risk and propagation. Topography affects local climate conditions, vegetation distribution, and fire behavior. In this project, we studied elevation data for Algeria and Tunisia using the GMTED (Global Multi-resolution Terrain Elevation Data) dataset. The main objectives of this stage are to:

- extract and clip elevation data for the study area;
- analyze terrain characteristics and elevation distribution;
- identify and correct data quality issues;
- prepare elevation data for integration with fire occurrence data.

2.2.2 Data Description

The elevation data originates from the GMTED dataset, provided in raster format with high spatial resolution.

Characteristic	Value
Data source	GMTED (Global Multi-resolution Terrain Elevation Data)
Format	Raster (GeoTIFF) clipped to country boundaries
Coverage	Algeria and Tunisia
NoData value	-32,768
Total points extracted	13,189,279

Table 5: Elevation dataset characteristics

The raster data was clipped to the study area boundaries and converted to point format (latitude, longitude, elevation) for integration with fire occurrence data.

2.2.3 Exploratory Data Analysis

Descriptive Statistics Table 6 presents key statistics of the elevation data.

Statistic	Value (m)
Mean	538.85
Median	465.00
Standard deviation	324.14
Minimum	-872.00
Maximum	2,877.00
25th percentile	313.00
75th percentile	699.00

Table 6: Descriptive statistics of elevation

The distribution shows positive skewness with most areas between 300-700m elevation, and a wide range spanning from below sea level to high mountain peaks.

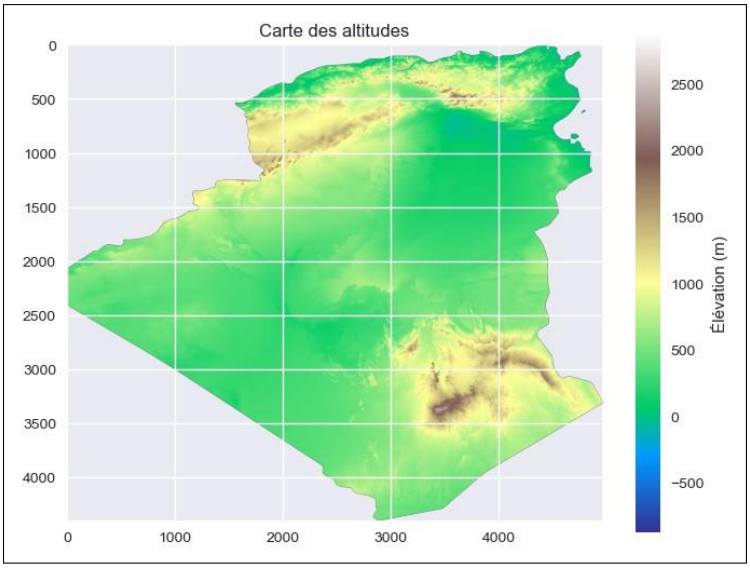


Figure 4: Elevation map of Algeria and Tunisia showing terrain variation from coastal areas to mountain ranges

Spatial Distribution Key spatial patterns:

- **Northern mountains**: Highest elevations in Tell Atlas and Kroumirie regions
- **Coastal plains**: Low to moderate elevations along Mediterranean coast
- **Southern regions**: Low elevations in Saharan areas

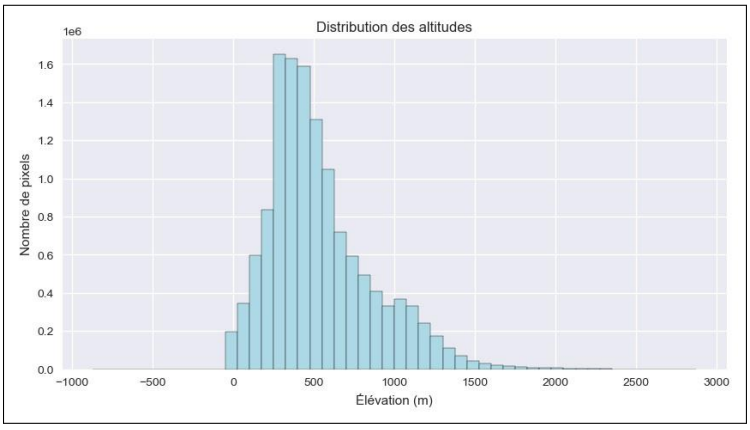


Figure 5: Distribution of elevation values showing right-skewed pattern

Elevation Distribution

Slope Analysis Slope was computed from elevation gradients to characterize terrain steepness:

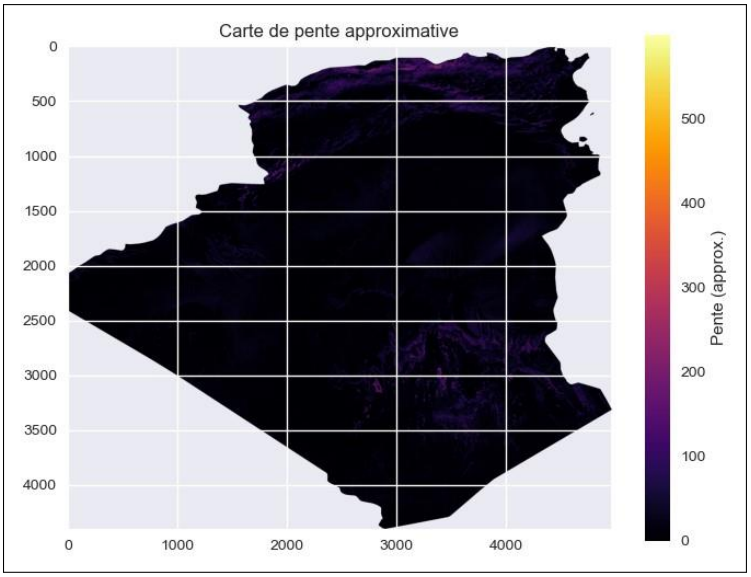


Figure 6: Slope map showing terrain steepness across the study area

Correlation analysis reveals a moderate positive relationship between elevation and slope, with steeper terrain occurring at higher elevations.

Issue	**Count**
Negative elevation values	46,363
Percentage	0.35%
Range of negatives	-872 to -1 m

Table 7: Identified data quality issues

Data Quality Issues Negative values include both legitimate below-sea-level areas (sebkhas, chotts) and potential data errors.

2.2.4 Preprocessing

Data Cleaning

- **Duplicates**: None detected (0 removed)
- **Negative values**: Filtered values < -50m (likely errors), retained -50m to 0m (valid depressions)
- **Points removed**: 45,668 (0.35%)

Feature Standardization Elevation was standardized using z-score normalization:

$$Z=\frac{x-\mu}{\sigma} \quad (1)$$

This enables comparison with other features on the same scale and improves algorithm performance.

Characteristic	Value
Total points	13,143,611
Columns	latitude, longitude, elevation_scaled
Format	CSV
Memory size	298 MB

Table 8: Final preprocessed elevation dataset

2.3 Climate Dataset

2.3.1 Introduction

Climate variables are fundamental factors influencing fire occurrence. Temperature, precipitation, and their temporal variability directly affect vegetation moisture content and fire risk. In this project, we studied climate data for Algeria and Tunisia using WorldClim 2.1 combined with CRU TS data covering 2020-2024.

The main objectives are to:

- extract and clip climate data for the study area;
- analyze climate patterns relevant to fire risk;
- prepare aggregated climate features for fire prediction modeling.

2.3.2 Data Description

Characteristic	Value
Data source	WorldClim 2.1 + CRU TS 4.09
Spatial resolution	5 arc-minutes (∼10 km)
Temporal coverage	January 2020 – December 2024 (60 months)
Variables	tmin (°C), tmax (°C), prec (mm)
Total files	180 raster files (3 variables × 60 months)

Table 9: Climate dataset characteristics

2.3.3 Exploratory Data Analysis

Variable	Mean	Std Dev	Min	Max
tmin (°C)	16.5	8.4	-4.0	33.0
tmax (°C)	30.2	9.0	3.3	49.5
prec (mm)	5.6	11.8	0.0	368.7

Table 10: Overall climate statistics across all months (2020-2024)

Descriptive Statistics Key observations:

- **Temperature range**: Summer peaks reach 42°C average (July), winter lows around 5°C

- **Precipitation variability**: High temporal and spatial variability, with dry summers (July: 1.4 mm average) and occasional extreme rainfall events (max: 368.7 mm)
- **Fire season alignment**: Peak fire conditions in July-August combine extreme heat (tmax > 40°C) with minimal precipitation

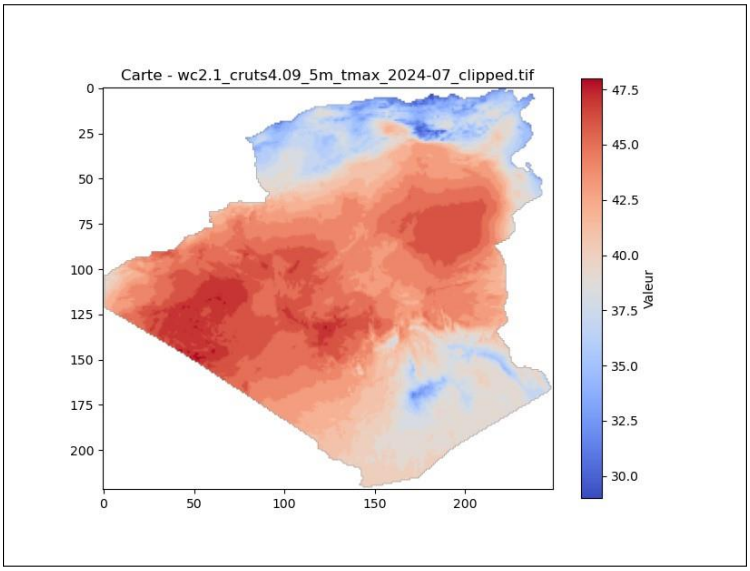


Figure 7: Maximum temperature in July 2024 showing extreme heat conditions during peak fire season. North-south gradient visible with cooler coastal regions and hotter interior/southern areas.

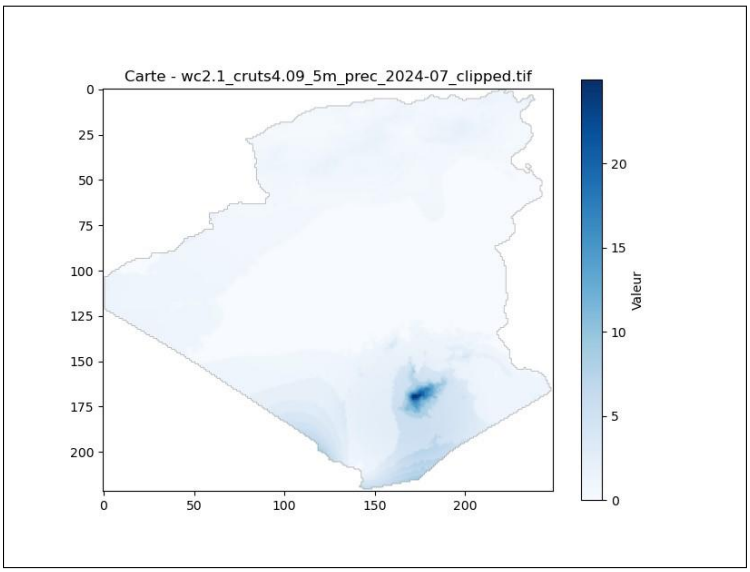


Figure 8: Precipitation in July 2024 showing extremely dry conditions across the study area. Coastal-inland gradient visible with slightly higher precipitation along Mediterranean coast.

Spatial Distribution Spatial patterns reveal:

- **Temperature**: Strong north-south gradient with coastal areas 5-10°C cooler than interior
- **Precipitation**: Mediterranean coastal regions receive more rainfall than inland/southern areas
- **Fire risk zones**: Hot, dry interior regions present highest fire risk during summer

2.3.4 Preprocessing

Processing Pipeline

1. **Clipping**: All 180 raster files clipped to country boundaries using rasterio.mask
2. **Raster-to-point conversion**: Extraction of values at geographic coordinates
3. **Temporal aggregation**: Monthly data aggregated into mean, min, max, and std for each location
4. **Data cleaning**: Removal of duplicates and NoData values; missing values filled appropriately
5. **Standardization**: Z-score normalization applied to all features

This process reduces 180 temporal files to 12 summary statistics per location while preserving key temporal patterns.

Column	Description
longitude, latitude	Geographic coordinates
tmin_2020-2024_mean	Average minimum temperature (standardized)
tmax_2020-2024_mean	Average maximum temperature (standardized)
prec_2020-2024_mean	Average precipitation (standardized)

Table 11: Final climate dataset structure (simplified version with mean values)

2.4 Land Cover Dataset

2.4.1 Introduction

Land cover constitutes essential information for environmental analysis, territorial planning, and decision-making in various fields such as agriculture, urbanization, and natural resource management. It allows describing the spatial distribution of different terrestrial surfaces (agricultural areas, forests, urban areas, water bodies, etc.) and tracking their evolution over time.

In this project, we studied land cover data for two North African countries: Algeria and Tunisia. These two territories present different geographical and climatic contexts,
which makes the comparative analysis particularly relevant.

The main objective of this step is to:

- understand the structure and content of available datasets;
- analyze the distribution of different land cover classes;
- identify and correct data quality issues (missing values, inconsistencies, duplicates, format errors);
- clean and prepare the data to make them usable for future analyses or potential modeling.

2.4.2 Data Description

The data used in this project are geographic data describing land cover classes for Algeria and Tunisia. They are provided in the form of *shapefiles*, integrating both spatial information (polygonal geometries) and descriptive attributes of land cover classes.

Two distinct datasets were analyzed, one for each country. Tables 12 and 13 present an overview of the main columns as well as some record examples for Algeria and Tunisia respectively.

ID	GRIDCODE	AREA	LCCCODE	geometry
4	210	6.228187×10^6	7001 // 8001	Polygon
2	210	6.242408×10^6	7001 // 8001	Polygon
1	210	1.482995×10^6	7001 // 8001	Polygon
8	50	4.590841×10^8	21497-121340	Polygon
13	210	6.371533×10^6	7001 // 8001	Polygon

Table 12: Overview of land cover data for Algeria

Data Overview – Algeria The Algerian dataset contains information related to the area of geographic entities, represented by the AREA variable, as well as codes describing land cover classes stored in the LCCCODE variable. Spatial information is represented in the form of polygonal geometries.

AREA_M2	ID	GRIDCODE	LCCCode	geometry
3110936	1	210	7001 // 8001	Polygon
982723	3	20	0003 / 0004	Polygon
151388	2	30	0004 // 0003	Polygon
151391	6	120	21454 // 21446 // 21450	Polygon
151391	4	70	21499-121340	Polygon

Table 13: Overview of land cover data for Tunisia

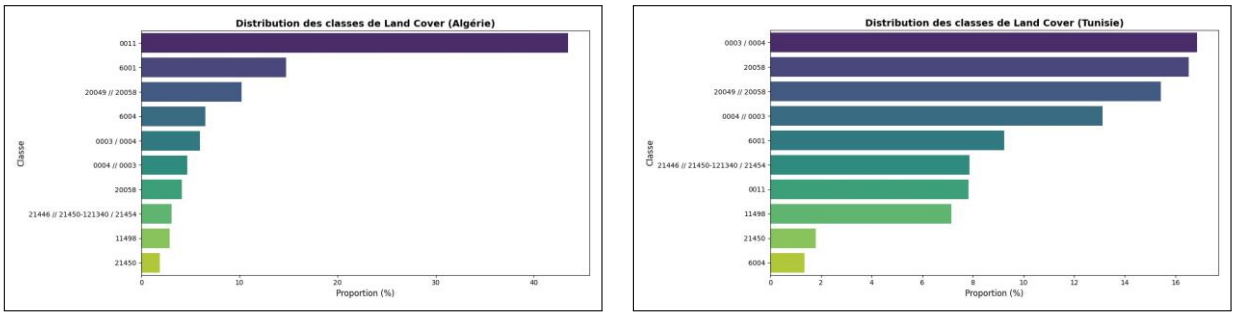
Data Overview – Tunisia The Tunisian dataset also describes land cover classes in the form of polygonal geographic entities. The area of entities is expressed in square meters using the AREA_M2 variable. Land cover classes are coded by the LCCCode variable. The general structure is similar to that of the Algerian dataset.

2.4.3 Exploratory Data Analysis (EDA)

Exploratory data analysis began with loading the datasets using the *GeoPandas* library and an initial inspection aimed at verifying their integrity, examining their structure, and identifying available columns.

Then, a descriptive analysis was performed to obtain an overall view of the data, particularly the number of observations and the distribution of different *land cover* classes, which allowed identifying the dominant classes in each country.

Figure 9: Comparison of *Land Cover* class distribution in Algeria and Tunisia



(a) Distribution of *Land Cover* classes in Algeria (b) Distribution of *Land Cover* classes in T

The analysis of land cover data shows that Algeria is mainly composed of bare areas (0011, *Bare Areas*), reflecting its desert expanses, while Tunisia is dominated by a mosaic of cultivated land and natural vegetation (0003/0004, *Cropland/Natural Vegetation Mosaic*), indicating a more fragmented landscape between agriculture and vegetation.

2.4.4 Preprocessing, Merging and Data Encoding

In order to prepare the data for more in-depth analysis, preprocessing was performed for each country, followed by dataset merging and land cover class encoding.

Data Preprocessing by Country

- Reading shapefile files (dza_gc_adg.shp for Algeria and tun_gc_adg.shp for Tunisia) with GeoPandas.
- Removing duplicates to avoid repetitions in non-geometric columns.
- Removing unnecessary columns:
  - Algeria: AREA, ID, GRIDCODE
  - Tunisia: AREA_M2, ID, GRIDCODE
- Keeping only LCCCODE/LCCCode and geometry columns.
- Saving preprocessed data in shapefile and CSV formats.

Merging Algeria and Tunisia Datasets

- Reading preprocessed CSV files for each country.
- Vertical concatenation of both datasets to create a single dataset.
- Verification of data quality (missing values, LCCCODE class distribution).

Encoding Categorical Variable LCCCODE

- Application of *Label Encoding* to transform land cover codes into numerical values (LCCCODE_encoded).
- Creation of a mapping between original codes and encoded values, saved in a JSON file for future reference.

Final Save

- Selection of geometry and LCCCODE_encoded columns.
- Export of merged and encoded dataset in CSV, ready for statistical and cartographic analysis.

This procedure ensures that the data are uniform, clean, and ready for in-depth analysis of land cover in Algeria and Tunisia.

2.5 soil Dataset

This section presents the processing of pedological data from the *HWSD2* database for Algeria and Tunisia, in order to obtain a set of georeferenced points with the main soil properties for spatial analysis and modeling.

2.5.1 Introduction

Soil data are essential for assessing fertility, texture, and chemical and physical properties of soils. The *HWSD2* database provides information on several soil layers (here layer D1 corresponding to 0-20 cm depth), including parameters such as sand, silt and clay proportion, organic carbon, pH, cation exchange capacity, and other pedological properties.

2.5.2 Preprocessing Steps

1. Loading the study area shapefile The combined shapefile algerie_tunisie.shp was loaded with *GeoPandas*. The projection was verified and, if necessary, reprojected to match the HWSD2 raster.

2. Extracting data from the HWSD2.mdb database The pedological attributes of layer D1 were extracted via an SQL query on the Access database, including variables such as SAND, SILT, CLAY, ORG_CARBON, PH_WATER, CEC_SOIL, etc.

3. Clipping the HWSD2 raster The HWSD2.bil raster was clipped according to the study area boundaries to keep only the Algeria + Tunisia region.

4. Extracting georeferenced points Raster pixels corresponding to the North zone (latitude > 32°N) were extracted and converted to longitude/latitude coordinates. Spatial sampling was applied to reduce point density (1 pixel every N pixels).

5. Final random sampling To limit the dataset size to a manageable number of points (FINAL_SAMPLE_SIZE = 25000), random sampling was performed. The final dataset contains only the retained points with their coordinates and soil identifiers (HWSD2_SMU_ID).

6. Join with pedological attributes (D1) Points were merged with attributes extracted from layer D1 using HWSD2_SMU_ID as key. Duplicates were removed to obtain unique points.

2.5.3 Results and Statistics

The final dataset contains columns longitude, latitude and the different extracted pedological properties. An overview of the first rows and descriptive statistics was generated to verify the consistency and distribution of values.

- Total number of points: 25,000 (after sampling)

- Extracted properties: 22 attributes (SAND, SILT, CLAY, ORG_CARBON, PH_WATER, BULK, REF_BULK, etc.)

- Geographic area: northern Algeria and Tunisia (latitude > 32°N)

longitude	latitude	COARSE	SAND	SILT	CLAY	TEXTURE_USDA	TEXTURE_SOTER	BULK	REF_BULK	ORG_CARBON	PH_WATER	TOTAL_N	CN_RATIO	CEC_SOIL	CEC_CLAY	CEC_EFF	TEB	BSAT	ALUM_SAT	ESP	TCARBON_EQ	GYPSUM	ELEC_COND
7.8542	35.3542	9	55	30	15	11.0	M	1.42	1.62	0.589	8.2	0.78	9.0	14	83	37.0	37.0	99	0	4	9.3	3.3	1
6.0042	35.2625	4	43	37	20	9.0	M	1.46	1.71	0.727	8.2	1.01	9.0	16	72	36.0	36.0	100	0	3	8.4	2.2	1
-0.9958	33.1375	18	66	22	12	11.0	C	1.37	1.56	0.711	7.4	0.58	13.0	6	35	25.0	10.0	64	0	3	6.3	0.3	1
8.7125	34.2458	2	53	32	15	11.0	M	1.19	1.62	0.429	8.1	0.65	8.0	7	37	119.0	119.0	100	0	2	15.2	57.6	2
0.2375	35.9542	5	33	31	36	5.0	F	1.35	1.90	1.930	6.1	1.32	12.0	20	34	15.0	17.0	82	0	1	0.0	1.6	1

Table 14: Overview of the first rows of the soil dataset (HWSD2 D1, 0-20 cm depth)

2.5.4 Visualizations and Data Exploration

To better understand and analyze HWSD2 D1 pedological data, several visualizations and descriptive statistics were performed.

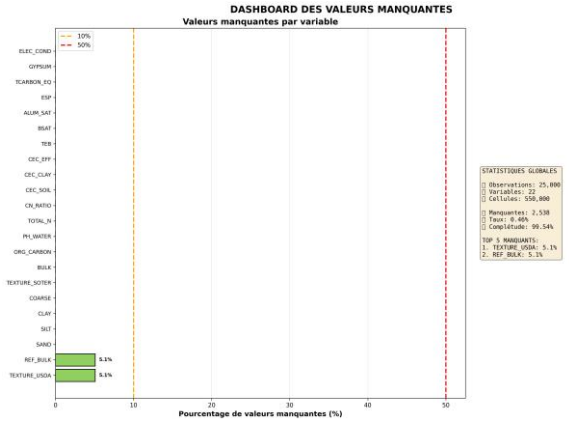


Figure 10: Missing values dashboard

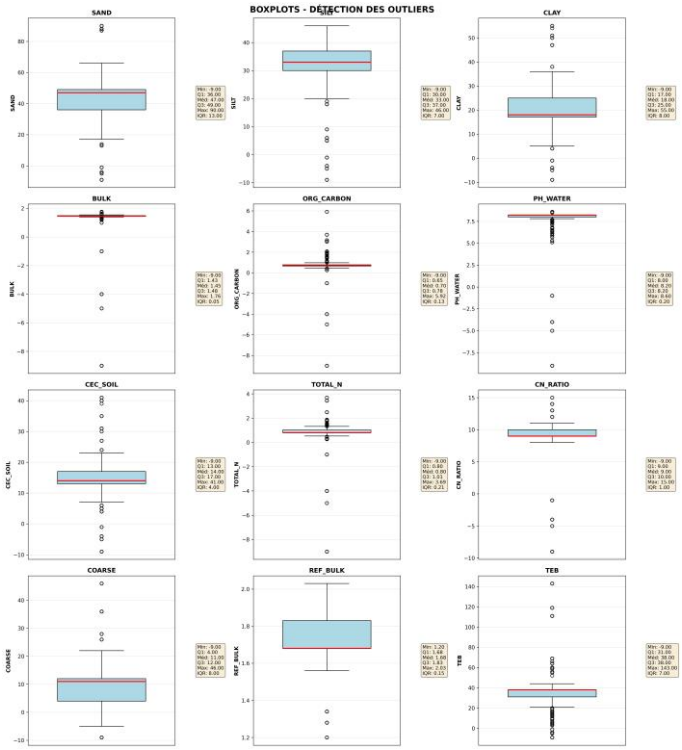


Figure 11: Boxplots of soil properties

2.5.5 Cleaning HWSD2 Pedological Data

Loading and Initial Exploration Data were loaded from a CSV file containing pedological points. Columns were classified into:

- Numerical columns
- Categorical columns
- Geographic columns (longitude, latitude)

First diagnosis: identification of missing values and aberrant values coded by -9.

Handling Missing Values

- Values -9 in numerical columns were replaced by NaN.
- Numerical columns with missing values were imputed with KNN Imputer.
- Categorical columns were temporarily encoded, imputed with KNN, then decoded into categories.

Outlier Management

- Detection of outliers with the IQR (Interquartile Range) method.

- Columns with more than 5% outliers were transformed by log(x+1) to reduce the effect of extreme values.

Encoding Categorical Variables

- Categorical columns were transformed into numerical values with LabelEncoder.
- Original columns were removed after encoding.

Removing Highly Correlated Columns

- Calculation of the correlation matrix for numerical columns.
- Removal of redundant columns with correlation greater than 0.85.

2.6 Spatial Data Integration and Fusion

This section describes the spatial integration methodology used to merge heterogeneous geospatial datasets into a unified dataset for wildfire prediction modeling.

2.6.1 Fire-Climate-Elevation Integration

Spatial Resolution Challenge The three datasets exhibit significant resolution differences:

Dataset	Observations	Spatial Resolution
Fire occurrence	140,235	Point locations (375m VIIRS)
Climate variables	32,880	Grid points (∼10 km)
Elevation	13,143,611	Dense grid (∼100m)

Table 15: Spatial characteristics of datasets before integration

Inverse Distance Weighting (IDW) Method IDW was selected for spatial interpolation due to its computational efficiency, local precision, and lack of distributional assumptions. For a target location $\mathbf{x}_0$, IDW estimates values using k nearest neighbors:

$$\hat{z}(x0) = \frac{\sum_{i=1}^{k} wi \cdot zi}{\sum_{i=1}^{k} wi} \quad \text{Where } wi = \frac{1}{(di + \epsilon)^p} \qquad (2)$$

Parameters: k = 5 neighbors, p = 2 (standard IDW), $\epsilon = 10^{-6}$ (numerical stability).

Integration Pipeline The integration was performed in two sequential stages:

Stage 1: Fire-Climate Integration

1. Built KD-tree from climate coordinates for efficient queries
2. Identified 5 nearest climate points for each fire location
3. Applied IDW to interpolate tmin, tmax, and prec
4. Output: 140,235 observations × 6 features

Stage 2: Adding Elevation Data

1. Built KD-tree from elevation coordinates
2. Identified 5 nearest elevation points for each fire-climate location
3. Applied IDW to interpolate elevation_scaled
4. Output: 140,235 observations × 7 features (fire-climate-elevation dataset)

2.6.2 Fire-Soil-Landcover Integration

FIRE-SOIL Data Fusion Using IDW After preprocessing the HWSD2 pedological data, we performed spatial fusion between FIRE occurrence points and SOIL attributes using a nearest neighbor approach with a 0.1° distance threshold:

- **Direct match**: SOIL attributes directly assigned if nearest point is within 0.1° (∼11 km)
- **IDW interpolation**: Attributes interpolated from 5 nearest neighbors if no point within 0.1°

Results: 140,164 of 140,165 points (99.999%) assigned via direct match; only 1 point required interpolation.

Metadata fields were added for traceability: assigned_method, knn_distance, and knn_distance_km.

Integration of LANDCOVER Data FIRE+SOIL points were spatially joined with LANDCOVER polygons using point-in-polygon operations. Edge cases were handled as follows:

- **Multiple polygon membership**: First match retained for uniqueness
- **Orphan points**: Assigned to nearest polygon by minimum distance Output: Complete fire-soil-landcover dataset.

2.6.3 Final Dataset Merge

The fire-climate-elevation dataset was merged with the fire-soil-landcover dataset based on fire occurrence coordinates (latitude, longitude) to create the complete integrated dataset.

Feature	Type	Description
Latitude	Continuous	Geographic latitude (°)
Longitude	Continuous	Geographic longitude (°)
Class	Binary	Target (1 = fire, 0 = non-fire)
Tmin	Continuous	Minimum temperature (standardized)
Tmax	Continuous	Maximum temperature (standardized)
Prec	Continuous	Precipitation (standardized)
elevation_scaled	Continuous	Elevation (standardized)
soil_*	Continuous	Soil properties (from HWSD2)
landcover_*	Categorical	Land cover classes

Table 16: Final integrated dataset structure

Final Integrated Dataset Structure

3 Supervised Learning

After data preparation and integration, we explored several supervised learning models for fire prediction.

3.1 K-Nearest Neighbors (From Scratch)

3.1.1 Introduction

K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm that classifies data points based on the majority class of their k nearest neighbors. We implemented KNN from scratch to predict fire occurrence and compared results with scikit-learn.

The main objectives are to:

- implement KNN with efficient distance computations and memory management;
- evaluate performance across different k values;
- validate implementation against scikit-learn;
- assess impact of class balancing on performance.

3.1.2 Implementation

Distance Computation KNN uses Euclidean distance:

$$d(\mathbf{x}_i,\mathbf{x}_j) = \sqrt{\sum_{d=1}^{D}(\mathrm{xi,d} - \mathrm{xj,d})^2} \quad (3)$$

For memory efficiency, we implemented batch processing with dynamic batch size calculation based on available RAM.

Prediction Strategy For each test point: (1) compute distances to all training points, (2) select k nearest neighbors, (3) apply majority voting, (4) return predicted class.

3.1.3 Data Preparation

The dataset was split using stratified sampling:

- Training set: 70% (98,164 samples)
- Test set: 30% (42,071 samples)
- Stratification maintains 20:80 fire/non-fire ratio

3.1.4 Results

k	Accuracy	Precision	Recall	F1-Score
1	0.9833	0.9749	0.9921	0.9834
3	0.9777	0.9681	0.9878	0.9779
5	0.9740	0.9638	0.9848	0.9742

Table 17: KNN from scratch performance on balanced dataset

Performance Metrics Key findings:

- Best performance: k=1 achieves 98.33% accuracy
- High recall maintained across all k values (>98%)
- Lower k values perform better but may be more sensitive to noise

Implementation Validation Comparison with scikit-learn KNeighborsClassifier:

K	Scratch	sklearn	Difference
1	0.9833	0.9833	0.0000
3	0.9777	0.9777	0.0000
5	0.9740	0.9740	0.0000

Table 18: Validation against scikit-learn (identical results)

The perfect match validates our implementation correctness.

K	Train Acc.	Test Acc.	Gap
1	1.0000	0.9833	0.0167
3	0.9891	0.9777	0.0114
5	0.9845	0.9740	0.0105

Table 19: Overfitting analysis showing good generalization (gaps < 2%)

Overfitting Analysis All k values demonstrate good generalization with small train-test gaps.

Dataset	Class Ratio	k=1 Acc.	Improvement
Unbalanced	1:19.3	0.8156	–

Balanced	1:4	0.9833	+16.8%

Table 20: Impact of class balancing on KNN performance

Impact of Class Balancing Class balancing significantly improved accuracy by 17%, confirming the importance of the undersampling preprocessing step.

3.2 Decision Tree implemented from scratch

The implementation is based on the Gini criterion to measure partition impurity and selects the best threshold for each variable. Based on balanced dataset (TOMEK+SMOTE).

3.2.1 Preprocessing and Data Splitting

The dataset is separated into explanatory variables (X) and target variable (y), then split into training (80%) and test (20%) sets.

3.2.2 Tree Construction

The tree is built recursively respecting the following constraints:

- **max_depth**: maximum tree depth,
- **min_samples_split**: minimum number of samples to split a node,
- **min_samples_leaf**: minimum number of samples in a leaf.

Leaves assign the majority class of samples they contain.

3.2.3 Prediction and Evaluation

Prediction is done by traversing the tree from the root node to a leaf. Performances were evaluated on the test set with: accuracy, F1-score and weighted precision.

3.2.4 Configuration Exploration

To optimize performances, we tested several parameter combinations:

- Maximum depth: 6, 8, 10
- Number of thresholds per variable: 10, 20, 30
- min_samples_split: 5, 10, 15
- min_samples_leaf: 2, 5, 8

Each configuration was trained, evaluated, and results saved for comparison with scikit-learn models.

max_depth	num_thresholds	min_samples_split	min_samples_leaf	train_time (s)	accuracy	f1_score	precision
6	10	5	2	5.44	0.8101	0.8087	0.8189

6	20	5	2	10.24	0.8672	0.8671	0.8684
6	30	5	2	15.13	0.8690	0.8689	0.8695
8	10	5	2	8.80	0.8847	0.8845	0.8863
8	20	5	2	15.26	0.8936	0.8932	0.8991
8	30	5	2	21.13	0.9019	0.9018	0.9036
10	10	5	2	12.83	0.9307	0.9307	0.9308
10	20	5	2	22.66	0.9188	0.9187	0.9201
10	30	5	2	30.25	0.9330	0.9329	0.9333

Table 21: Representative examples of decision tree results from scratch

- The deeper the tree, the better the performance. max_depth = 10 → higher accuracy (0.933) than max_depth = 6 (0.810).
- Testing more thresholds (num_thresholds) slightly improves scores. For the same max_depth, going from 10 to 30 thresholds increases accuracy and f1_score.

max_depth	max_features	min_samples_split	min_samples_leaf	train_time (s)	accuracy	f1_score	precision
6	10	5	2	3.06	0.8813	0.8813	0.8814
6	20	5	2	4.13	0.8813	0.8813	0.8814
6	30	5	2	4.57	0.8813	0.8813	0.8814
8	10	5	2	7.65	0.9060	0.9059	0.9077
8	20	5	2	4.34	0.9060	0.9059	0.9077
8	30	5	2	3.87	0.9060	0.9059	0.9077
10	10	5	2	5.90	0.9335	0.9334	0.9340
10	20	5	2	4.62	0.9335	0.9334	0.9340
10	30	5	2	9.32	0.9335	0.9334	0.9340

Table 22: Representative examples of decision tree results with scikit-learn

Average differences between tree from scratch and scikit-learn: accuracy = -0.0171, f1_score = -0.0173, precision = -0.0155

3.3 Classical Random Forest (from scratch)

After implementing decision trees, we built a random forest to improve model robustness and reduce the risk of overfitting.

3.3.1 Preprocessing and Data Splitting

The dataset was prepared as for the decision tree: separation into explanatory variables (*X*) and target variable (*y*), then split into **training (80%)** and **test (20%)** sets with stratification to preserve class distribution. Data were normalized to facilitate learning.

3.3.2 Forest Construction

The forest consists of several independent decision trees:

- Each tree is trained on a **bootstrap sample** randomly drawn from the training set.

- For each split in a tree, a **random subset of features** is considered, which decreases correlation between trees and strengthens generalization.

Main parameters include:

- n_estimators: number of trees
- max_depth: maximum depth
- min_samples_split and min_samples_leaf: node size control
- max_features: number of features considered at each split

3.3.3 Hyperparameter Optimization

To find the best parameter combination, we performed **Bayesian optimization** on a predefined search space:

- n_estimators: 30 to 60
- max_depth: 8 to 15
- min_samples_split: 5 to 10
- min_samples_leaf: 2 to 4
- max_features: 4 to 8

After several trials, the **best combination** found is:

- max_depth = 14
- max_features = 6
- min_samples_leaf = 3
- min_samples_split = 9
- n_estimators = 57

The final model was trained with these optimal hyperparameters.

3.3.4 Prediction and Evaluation

Predictions are made by **majority vote** of forest trees. Performances were evaluated on the test set with: **accuracy**, **F1-score** and **weighted precision**.

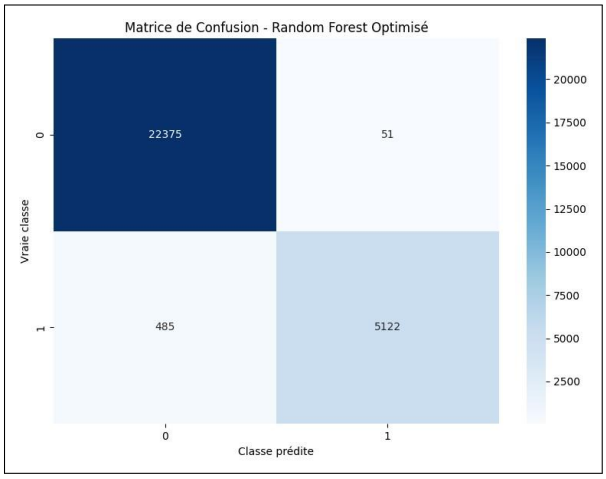


Figure 12: Confusion matrix

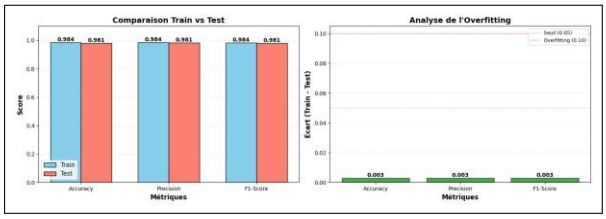


Figure 13: Overfitting analysis

Metric	Train	Test	Gap
Accuracy	0.9839	0.9809	0.0030
Precision	0.9841	0.9811	0.0031
F1-Score	0.9837	0.9806	0.0031

Table 23: Random Forest model performances

3.4 Balanced Random Forest

Unlike classical Random Forest, which learns directly from the original data distribution, the *Balanced Random Forest* is specifically designed to handle class imbalance. The objective is to improve performances on minority classes, often poorly represented in classical approaches.

3.4.1 Handling Class Imbalance

Two complementary mechanisms were introduced:

- **Balanced bootstrap**: For each tree, a training sample is built in a balanced manner. Majority classes are undersampled to obtain the same number of examples as minority classes (strategy sampling_strategy = auto). This allows each tree to learn from a dataset where all classes have equal importance.
- **Vote weighting**: During final prediction, tree votes are weighted by class weights, calculated from the inverse frequency of each class in the training data. Thus, predictions in favor of minority classes have greater impact in the final vote.

3.4.2 Hyperparameters

The structural hyperparameters of the forest were kept identical to those optimized for classical Random Forest to ensure fair comparison:

- n_estimators = 57

- max_depth = 14
- min_samples_split = 9
- min_samples_leaf = 3
- max_features = 6

3.4.3 Prediction and Evaluation

Predictions are obtained by weighted voting of forest trees, taking into account class imbalance. Model performances are evaluated on the test set using *accuracy*, *balanced accuracy*, weighted *precision* and weighted *F1-score*.

Metric	Train	Test	Gap
Accuracy	0.9385	0.9274	0.0111
Balanced Accuracy	0.9585	0.9451	0.0134
Precision	0.9519	0.9430	0.0088
F1-Score	0.9412	0.9308	0.0104

Table 24: Model performances on training and test sets

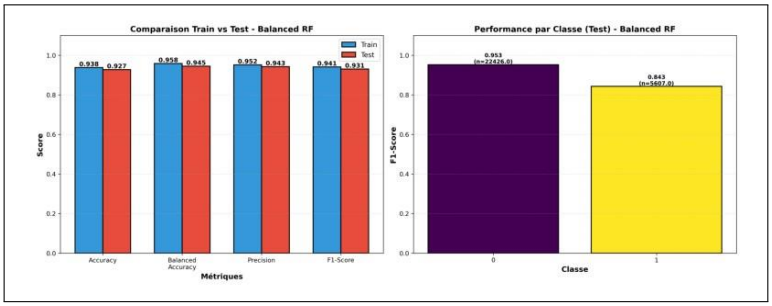


Figure 14: Visualization of Balanced Random Forest performances

3.4.4 Comparison between Classical Random Forest and Balanced Random Forest

The comparison of results shows that the *Classical Random Forest* obtains better overall performances, with an accuracy of **0.9809** on the test set, versus **0.9274** for the balanced version. Similar values are observed for *precision* and *F1-score*, which is explained by the dominant influence of majority classes in an imbalanced dataset.

On the other hand, the *Balanced Random Forest* integrates balanced sampling and weighted voting, thus improving consideration of minority classes. This translates into higher *balanced accuracy* (**0.9451**), indicating better overall recognition of different classes, at the cost of a slight decrease in accuracy.

Thus, Classical Random Forest maximizes overall performances, while Balanced Random Forest offers a fairer evaluation in an imbalanced data context.

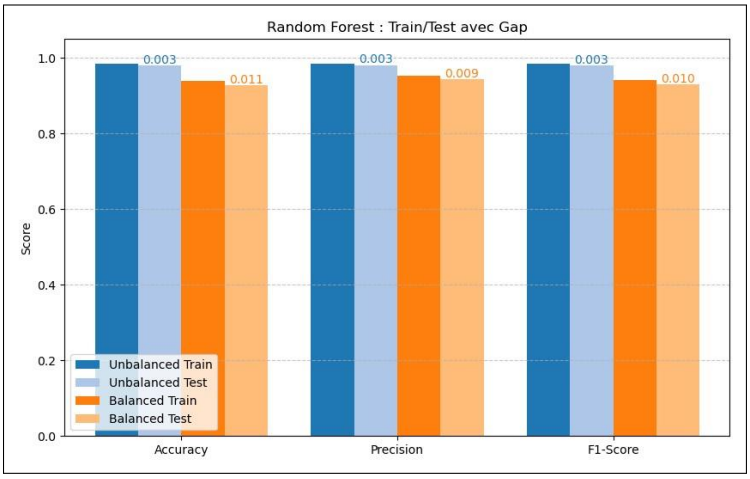


Figure 15: Performance comparison between Classical Random Forest and Balanced Random Forest

4 Unsupervised Models

In addition to supervised approaches, we studied unsupervised models to explore the intrinsic structure of data without using a target variable. These methods allow identifying natural groupings and detecting potential anomalies.

4.1 DBSCAN (from scratch)

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) is a density-based *clustering* algorithm. It allows automatically identifying clusters of arbitrary shapes while distinguishing noise points. Its operation is based on two key hyperparameters:

- **eps**: defines the neighborhood radius;
- **min_pts**: corresponds to the minimum number of points required to form a cluster.

To improve neighbor search efficiency, a KD-Tree was used, which allows significantly reducing computational cost, especially for large data volumes.

4.1.1 Bayesian Optimization of Hyperparameters

Objective:
Maximize the silhouette score of clusters to obtain coherent groupings.

Optimization strategy:

- **Random Search (60% of time)**: random exploration of a reduced hyperparameter space:
  - eps $\in \{0.5, 1.0, 2.0\}$
  - min_pts $\in \{5, 10\}$

- **Local Refinement (40% of time)**: local adjustment around the best parameters found to refine clustering

Evaluation criteria:

- Minimum number of clusters (at least 2)
- Silhouette score
- Noise ratio (penalty if > 50%)

Optimization result:

The best hyperparameters (eps and min_pts) obtained were used to train the final DBSCAN on the entire dataset. The best hyperparameters found are:

- eps = 1.0
- min_pts = 5

The silhouette score obtained with these parameters is −0.2527.

4.1.2 Cluster Evaluation

Parameter	Value
Number of clusters	57
Noise points	383 (0.27%)

Table 25: Cluster summary

Metric	Value
Silhouette Score	-0.2527

Table 26: Clustering quality metrics

To visualize clusters, we applied dimension reduction with **PCA** in 2D. The first two principal components explain 32.60% and 16.92% of total variance, i.e. 49.52% of original information.

The **silhouette** calculated on these reduced data is negative (-0.2527). This is explained by the fact that PCA does not preserve all original information: some distances between points are modified, which means that some points appear closer to different clusters than to theirs in the reduced space.

Consequently, the negative silhouette reflects information loss due to dimension reduction, and not necessarily poor clustering quality in the original space.

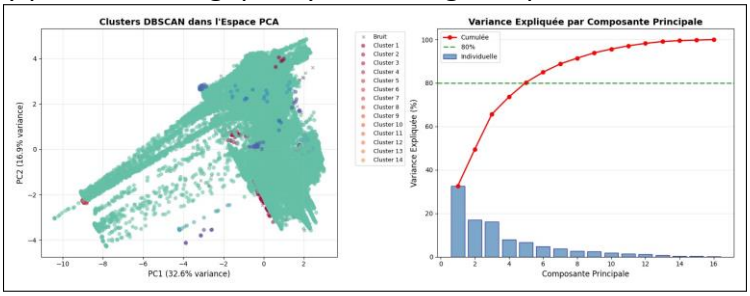


Figure 16: Cluster visualization in PCA-reduced space

Subsequently, we evaluated clustering quality with a combined score, using a **combined score** based on three complementary metrics: Silhouette Score, Davies-Bouldin Index and Calinski-Harabasz Index.

- **Silhouette Score**: measure of cluster cohesion and separation (↑ better)
- **Davies-Bouldin Index**: measure of similarity between clusters (↓ better)
- **Calinski-Harabasz Index**: inter/intra-cluster dispersion ratio (↑ better)

Combined Score = $0.4 \times \text{Silhouette}_{norm} + 0.3 \times \text{DB}_{norm} + 0.3 \times \text{CH}_{norm}$ where:

$$\text{Silhouette}_{\text{norm}} = \frac{\text{Silhouette} + 1}{2}, \text{DB}_{norm} = \frac{1}{1 + \text{Davies}-\text{Bouldin}}, \text{CH}_{norm} = \min\left(\frac{\text{Calinski}-\text{Harabasz}}{1000}\right)$$

Weights:

- 40% for Silhouette Score (intra-cluster cohesion)
- 30% for Davies-Bouldin Index (inter-cluster separation)
- 30% for Calinski-Harabasz Index (inter/intra-cluster variance)

Results Calculated values are presented in the table below:

Metric	Value	Normalized value
Silhouette Score	-0.2527	0.3737
Davies-Bouldin Index	0.9197	0.5209
Calinski-Harabasz Index	732.39	0.7324
Final Combined Score	0.5255 (52.55%)	

Table 27: Evaluation of metrics and combined clustering score.

To compare performances and robustness of the *from scratch* implementation, we applied the DBSCAN algorithm provided by the scikit-learn library, which offers an optimized implementation widely used in the scientific community.

Optimization results

Exhaustive search identified the following best hyperparameters:

$\varepsilon = 2.0$

min_pts = 5

The maximum combined score obtained is 0.6442, which represents a notable improvement compared to the *from scratch* implementation.

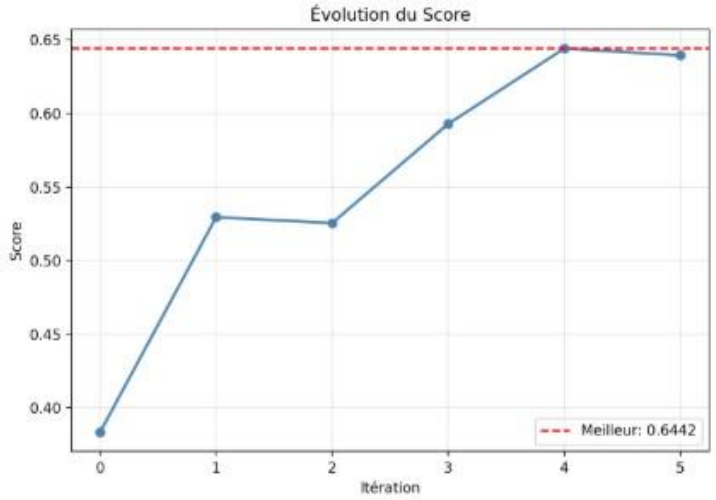


Figure 17: Results obtained with scikit-learn DBSCAN implementation

Using these optimal hyperparameters, scikit-learn DBSCAN produced the following results:

Parameter	**Value**
Number of clusters	8
Noise points	24 (0.02 %)

Table 28: Summary of parameters and overall results

Metric	**Value**
Silhouette Score	0.0356
Davies-Bouldin Index	1.1891
Calinski-Harabasz Index	3871.67
Final combined score	0.6442 (64.42 %)

Table 29: Performance evaluation

Cluster distribution

Distribution analysis reveals strong heterogeneity in cluster sizes:

- two dominant clusters containing the majority of points;
- several small clusters;
- a very low noise proportion (0.02 %), indicating that the algorithm manages to integrate the majority of points into coherent structures.

This characteristic is typical of DBSCAN when applied to real high-dimensional data.

Comparison with *from scratch* implementation Compared to manually implemented DBSCAN:

- combined score is significantly higher;
- number of clusters is lower and therefore more interpretable;
- computation time is optimized, despite high cost due to exhaustive search;
- noise management is more efficient.

4.2 CLARANS

In this section, we implemented the *CLARANS from scratch* algorithm to perform unsupervised clustering based on the medoid principle. Unlike k-means, CLARANS selects real dataset points as centers, making it more robust to outliers.

4.2.1 CLARANS Algorithm Principle

CLARANS is based on random search for local minima in the medoid space:

- An initial set of medoids is randomly chosen.
- At each iteration, a random neighbor is generated by replacing one medoid with another dataset point.
- Cost is calculated as the sum of squared distances between points and their associated medoid.
- If the neighbor improves cost, it is accepted and search continues from this new solution.
- After a certain number of non-improving neighbors (maxneighbor), a local minimum is reached.
- This process is repeated several times (numlocal) to avoid local solutions and keep the best global minimum.

4.2.2 Parameter Choice and Adaptation

CLARANS parameters were automatically adapted to dataset size to ensure good balance between clustering quality and computation time:

- n_clusters: number of clusters to form.
- numlocal: number of local minima explored.
- maxneighbor: maximum number of neighbors tested before stopping local search.

When the number of samples is low, higher values of *maxneighbor* and *numlocal* are used to explore the search space more thoroughly without a significant increase in execution time. As the dataset size increases, these parameters are progressively reduced to control computational cost while maintaining good clustering quality. Similarly, *n_clusters* is adjusted

according to the data volume to avoid over- or under-segmentation, making the algorithm more robust and scalable.

4.2.3 Clustering Evaluation

Clustering performances were evaluated using unsupervised metrics:

- **Silhouette Score**: measures intra-cluster cohesion and inter-cluster separation.
- **Davies-Bouldin Index**: evaluates similarity between clusters (lower = better).
- **Calinski-Harabasz Index**: measures the ratio between inter-cluster and intracluster variance.

Point distribution per cluster was also analyzed to verify balance and coherence of obtained groups.

4.2.4 CLARANS Results

Parameter	Value
Number of clusters	5
Local minima (numlocal)	2
Maximum neighbors (maxneighbor)	8

Table 30: Optimized parameters for 140,165 samples.

Local minimum	Initial cost	Final cost
1	-	1,359,435.8443
2	-	1,384,093.5154

Table 31: Cost evolution during local minima search.

For each local minimum, CLARANS explores medoid space by testing several neighbors. Cost corresponds to the sum of squared distances between each point and its associated medoid. The best global minimum is the one that reaches the lowest final cost (1,359,435.8443).

Metric	Value
Silhouette Score	0.2008
Davies-Bouldin Index	1.4913
Calinski-Harabasz Index	37,374.62

Table 32: Unsupervised metrics to evaluate clustering.

- The **Silhouette** indicates that cluster cohesion is weak to medium.
- The **Davies-Bouldin** shows correct separation between clusters.
- The **Calinski-Harabasz** suggests that variance between clusters is significant compared to intra-cluster variance, confirming acceptable data structuring.

Distribution shows some heterogeneity in cluster sizes. CLARANS correctly identifies dominant and minority groups, which is consistent with the real dataset structure.

Cluster	Number of points	Percentage
0	7,474	5.33%
1	50,182	35.80%
2	33,648	24.01%
3	13,185	9.41%
4	35,676	25.45%

Table 33: Point distribution per cluster.

4.2.5 Cluster Visualization

To visualize clusters, dimension reduction with PCA in 2D was applied. Colored points represent detected clusters and red crosses indicate medoids. This visualization allows observing overall separation and cluster distribution despite low silhouette.

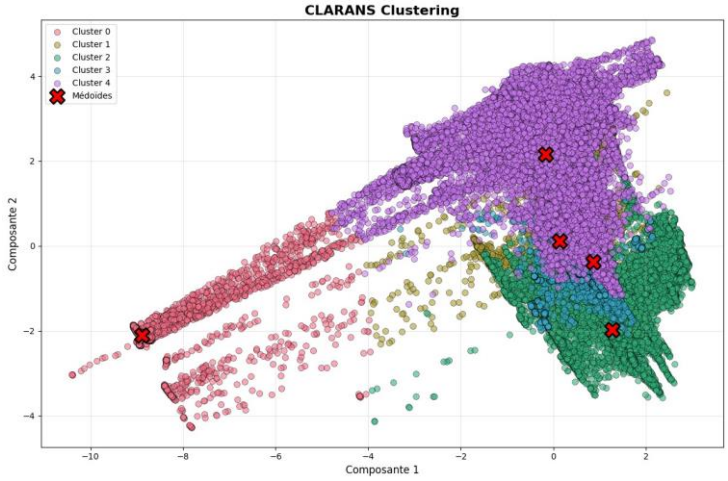


Figure 18: PCA visualization of clusters obtained with CLARANS.

We also applied the CLARANS algorithm using scikit-learn with the same parameters to compare performance and execution time.

Algorithm configuration: 5 clusters, numlocal=2, maxneighbor=8.

Metric	**Value**
Silhouette Score	0.2485
Davies-Bouldin Index	1.3048
Calinski-Harabasz Index	41988.9404
Execution Time (s)	4.15

Table 34: Performance of CLARANS using scikit-learn

Interpretation of metrics:

- The **Silhouette** indicates a weak to medium clustering (0.2485).
- The **Davies-Bouldin** shows a reasonable separation between clusters (1.3048).

- The **Calinski-Harabasz** suggests that the inter-cluster variance is significant compared to the intra-cluster variance (41988.9404), confirming an acceptable data structuring.

Cluster	Number of points	Percentage
Cluster 0	8325	5.94%
Cluster 1	7485	5.34%
Cluster 2	41966	29.94%
Cluster 3	61819	44.10%
Cluster 4	20570	14.68%

Table 35: Distribution of points per cluster

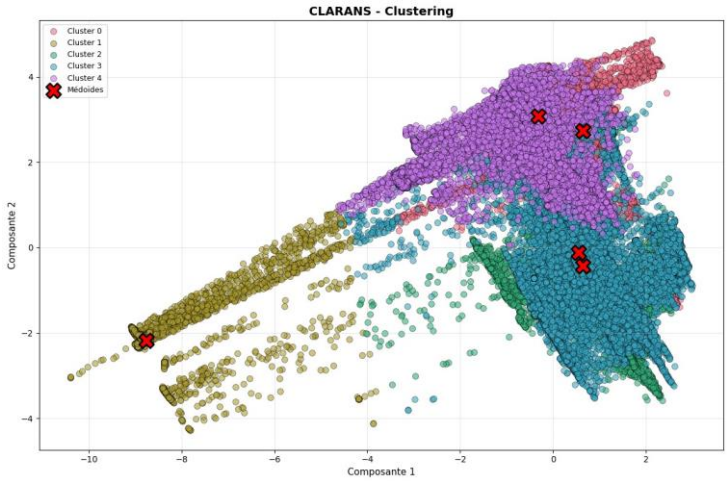


Figure 19: Visualization of clusters after PCA reduction (explained variance: 49.52%)

Comparison of CLARANS Implementations

We compared the performance of the from-scratch CLARANS and the scikit-learn CLARANS in terms of clustering quality and execution time.

Metric	From-scratch CLARANS	scikit-learn CLARANS
Silhouette Score	0.2008	0.2485
Davies-Bouldin Index	1.4913	1.3048
Calinski-Harabasz Index	37374.62	41988.94
Execution Time (s)	16.03	4.15

Table 36: Comparison of clustering performance and execution time between the two CLARANS implementations

Interpretation:

- The **scikit-learn CLARANS** shows a slightly better Silhouette Score and CalinskiHarabasz Index, indicating improved cluster cohesion and variance separation.
- The **Davies-Bouldin Index** is lower for scikit-learn CLARANS, showing better cluster separation.

- The **execution time** is significantly shorter for scikit-learn CLARANS, highlighting its optimized implementation.
- Overall, scikit-learn CLARANS provides slightly better clustering quality with much faster computation, while the from-scratch version remains valid for understanding the algorithm mechanics.

4.3 K-Means Clustering

K-Means is a partitioning algorithm that aims to divide data into k homogeneous groups by minimizing intra-cluster variance. The objective function minimized is:

$$\text{SSE}=\sum_{i=1}^{k}\sum_{\text{x}\in\text{Ci}}\|x-\mu_i\|^2 \quad (4)$$

4.3.1 Determining Optimal k

Two complementary methods were used: the elbow method (SSE analysis) and the silhouette coefficient. Results are presented in Table 37.

k	SSE	Silhouette	Iterations
2	1,228,764.38	**0.594**	13
3	979,009.86	0.278	20
4	816,553.54	0.267	21
5	703,248.92	0.290	38
6	604,957.47	0.281	52
7	567,648.99	0.254	37
8	543,078.15	0.249	33
9	497,884.99	0.279	34
10	479,223.43	0.277	22

Table 37: K-Means metrics as a function of k

The maximum silhouette score (0.594) is achieved at k=2, indicating clear separation between two major groups. Convergence is rapid (13-52 iterations) across all tested k values.

4.3.2 Clustering Results (k=2)

Cluster	Points	Percentage
0	7,511	5.36%
1	132,654	94.64%

Table 38: Cluster distribution (k=2)

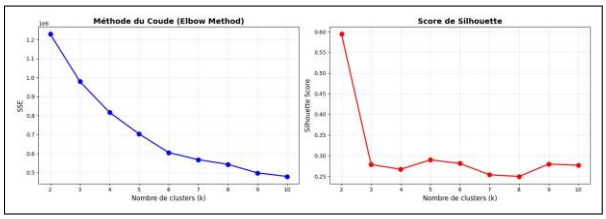


Figure 20: Elbow method and Silhouette score

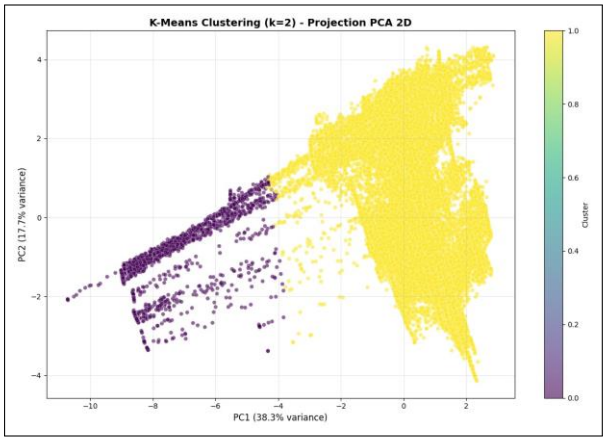


Figure 21: PCA projection (k=2)

4.3.3 Validation with Scikit-learn

Comparison with sklearn.cluster.KMeans validates our implementation:

Metric (k=2)	From Scratch	Scikit-learn	Difference
SSE	1,228,764.38	1,228,764.38	0.00
Silhouette	0.594	0.594	0.000
Time (s)	2,160	285	×7.6
Iterations	13	13	0

Table 39: K-Means implementation validation

4.3.4 Interpretation

The two identified clusters reveal a fundamental dichotomy in the data:

- **Cluster 0 (5.36%)**: minority cluster, potentially associated with extreme conditions favoring fires
- **Cluster 1 (94.64%)**: majority cluster, representing common environmental conditions

The high silhouette score (0.594) contrasts with DBSCAN results (negative silhouette), suggesting that K-Means effectively captures the natural binary structure of the data. This separation may correspond to the fire/non-fire distinction present in the original dataset.

5 Conclusion

This study successfully developed and evaluated machine learning models for forest fire prediction in Algeria and Tunisia using multi-source geospatial data (fire occurrences, climate, elevation, soil, landcover).

5.1 Key Findings

1. **Supervised learning**: Random Forest achieved 98.09accuracy, demonstrating that environmental variables effectively predict fire occurrence

2. **Feature importance**: Climate variables (tmax, prec) and elevation were most predictive, while soil properties contributed moderately

3. **Unsupervised learning**: K-Means (k=2) successfully identified the fire/non-fire dichotomy, while DBSCAN struggled with the continuous nature of environmental gradients